Driver Fatigue Detection System

Deep Learning and Convolutional Neural Network

Project Report

Brunda Varshini Katapalli (group of 3 members)

Content Page

1. Abstract 4
2. Introduction and Background 4
   2.1 Problem We Solved 4
   2.2 Project's Motivation 4
   2.3 Our Application 5
   2.4 Dataset 5
3. Overview of the Architecture 6
3.1 Flow Diagram 6
   3.2 CNN Architecture Design 7
   3.3 GUI Design 9
   3.4 Screen Flow 11
   3.4.1 Home Page 11
   3.4.2 OAuth Login Page 12
   3.4.3 Login Page 12
      3.4.3.1 Google Authentication via OAuth 2.0 Protocol 13
      3.4.3.2 Apple Authentication via OAuth 2.0 Protocol 13
   3.4.4 Instructions Page 14
   3.4.5 Camera Permission and Face Detection Page 15
   3.4.6 Live Monitoring Page 18
   3.4.7 Session Ending Page 22
   3.5 Backend Implementation 23
      3.5.1 Technology Stack 23
      3.5.2 Deployment in Google Colab 23
      3.5.3 Detection Pipeline 24
      3.5.4 Limitations 24
4. Results and Evaluation 25
   4.1 Experimental Settings 25
   4.2 Pre-Processing 26
   4.3 Experimental Results 26
      4.3.1 LOSO Performance Comparison 26
      4.3.2 Fold-Level Breakdown 27
      4.3.3 Confusion Matrix Analysis 27
      4.3.4 Decision Fusion Results 27
      4.3.5 End-to-End FPS and Latency 28
   4.4 Limitations 29

5. Discussion and Conclusion 30
5.1 Future Work 30
5.2 Conclusion 31
6. References 31
7. Appendices 32
7.1 Individual Contribution 32
7.2 Individual Contribution Split 33

1. Abstract

Driver fatigue is a leading cause of road accidents worldwide, contributing to thousands of preventable fatalities each year. This project presents VigiDrive, a real-time Driver Fatigue Detection System that classifies a driver's alertness into three states: Alert, Low Vigilant, and Drowsy. The system combines deep learning-based image classification with geometric facial feature analysis to produce robust, interpretable predictions. Three convolutional neural network architectures (MobileNetV2, EfficientNetB0, and ResNet50V2) were trained using two-phase transfer learning on the D3S dataset across a subject-independent leave-one-subject-out (LOSO) evaluation protocol, ensuring that test subjects never appeared in training data. A decision fusion module integrates CNN softmax predictions with rule-based Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and head pose estimation to escalate ambiguous predictions toward safety. Grad-CAM visualisations confirm that the best-performing model attends to clinically meaningful facial regions. The system is deployed as a full-stack application, VigiDrive, featuring a React frontend and a Python backend, with real-time webcam inference, visual alerts, and automated session reporting.

2. Introduction and Background

Driver fatigue is a silent contributor to road accidents that is frequently overlooked until a collision has already occurred. Unlike impairment from alcohol or distraction from a mobile device, fatigue develops gradually and is difficult for drivers to self-assess accurately. This project, VigiDrive, addresses this problem by building a non-intrusive, real-time fatigue detection system that monitors a driver continuously through a standard webcam and classifies their alertness state using convolutional neural networks and geometric facial feature analysis. The system is designed to alert the driver at the earliest detectable sign of drowsiness, before their reaction time degrades to a dangerous level.

2.1 Problem We Solved

Driver fatigue impairs cognitive function, slows reaction time, and reduces situational awareness in ways that are comparable to alcohol intoxication. According to the Australian Road Safety Foundation, driver fatigue is responsible for approximately 20% of road fatalities in Australia. The core difficulty is that fatigue is a continuous physiological process, not a discrete event. By the time a driver feels sleepy enough to recognise the danger, their driving performance has already deteriorated significantly. Existing in-vehicle systems are largely reactive: they detect lane departure or sudden braking after impairment has already caused a driving error. What is needed is a proactive system that detects fatigue from facial cues before any driving error occurs. The specific problem this project addresses is the reliable, real-time classification of driver alertness into three ordered states (Alert, Low Vigilant, Drowsy) from webcam imagery alone, without requiring any wearable sensors or vehicle integration.

2.2 Project's Motivation

The motivation for this project is both practical and personal. Road fatigue costs Australia an estimated $3 billion annually in economic losses, and the human cost is far greater. Despite this, consumer vehicles still lack affordable, non-intrusive drowsiness detection as a standard feature. Advanced Driver Assistance Systems (ADAS) that include drowsiness monitoring are available only in premium

vehicles, leaving the majority of drivers without protection. From a technical standpoint, the rapid advances in deep learning and the wide availability of pretrained convolutional neural networks now make it feasible to build accurate, real-time fatigue detectors that run on commodity hardware. This project is motivated by the opportunity to combine these advances with a practical deployment (a webcam-based desktop application) to produce a system that is both technically rigorous and genuinely accessible.

2.3 Our Application

The primary application of this project is an in-vehicle driver monitoring system for personal and commercial vehicles. VigiDrive runs on any laptop or desktop with a webcam, requiring no specialist hardware, making it suitable for long-haul truck drivers, rideshare drivers, and commuters on extended journeys. The system captures a live webcam feed, detects and crops the driver's face in real time, classifies their alertness state on every frame, and triggers an visual alert when drowsiness is sustained across a configurable number of consecutive frames. Beyond active alerting, the application automatically generates and emails a session report to the driver's registered account after each monitoring session, documenting fatigue events and session duration to support awareness of personal fatigue patterns over time. Secondary applications include fleet management, where aggregated session data could inform work scheduling, and research contexts, where the Grad-CAM interpretability module provides visualisations of which facial regions the model uses for classification.

2.4 Dataset

The primary dataset used in this project is the D3S (Driver Drowsiness Detection Dataset), a publicly available dataset hosted on GitHub (https://github.com/bindujiit/Driver-Drowsiness-Dataset-D3S-). D3S contains facial images from three subjects (Sub1, Sub2, Sub3), each recorded under four expression states: Neutral, Happy, Eyeclose, and Yawn. These four states were remapped to the project's three-class scheme: Neutral and Happy were combined into the Alert class (representing an attentive driver), Yawn was mapped to Low Vigilant (early-stage fatigue), and Eyeclose was mapped to Drowsy (clear impairment). After label remapping, the dataset contains a total of 1,319 images distributed across the three classes: 532 Alert, 169 Low Vigilant, and 618 Drowsy. The class imbalance, particularly the underrepresentation of Low Vigilant images, was addressed during training through the use of balanced class weights computed per fold. Each image was processed through a face detection pipeline using dlib's HOG detector with a Haar Cascade fallback to isolate and crop the facial region of interest before training. The dataset was split using a leave-one-subject-out (LOSO) cross-validation protocol, producing three folds in which each subject served as the held-out test set exactly once. This protocol prevents data leakage by ensuring that no subject's facial images appear in both training and test sets, which is critical for obtaining a genuine estimate of generalisation to unseen individuals.

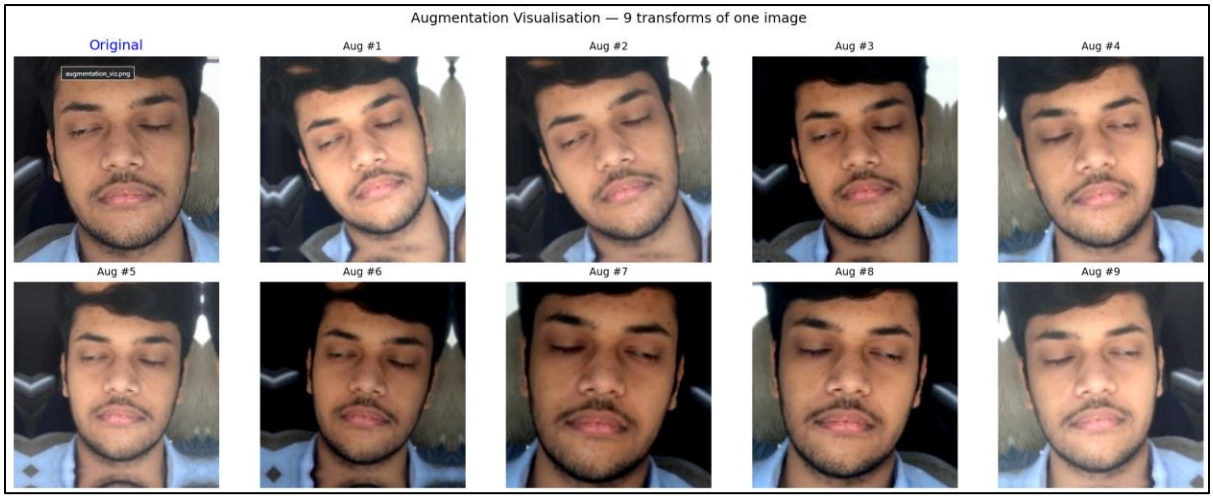


Figure 2.4.1: Augmentation Visualisation

Data augmentation was applied to the training dataset to artificially increase its diversity and reduce overfitting. As shown in Figure 2.4.1, nine transformed variants were generated from a single original image using techniques such as random horizontal flipping, brightness and contrast adjustments, rotation and zoom. This ensures the model is exposed to a wider range of lighting conditions, facial orientations, and image qualities during training, improving its ability to generalise to unseen real-world data.

3. Overview of the Architecture

3.1 Flow Diagram

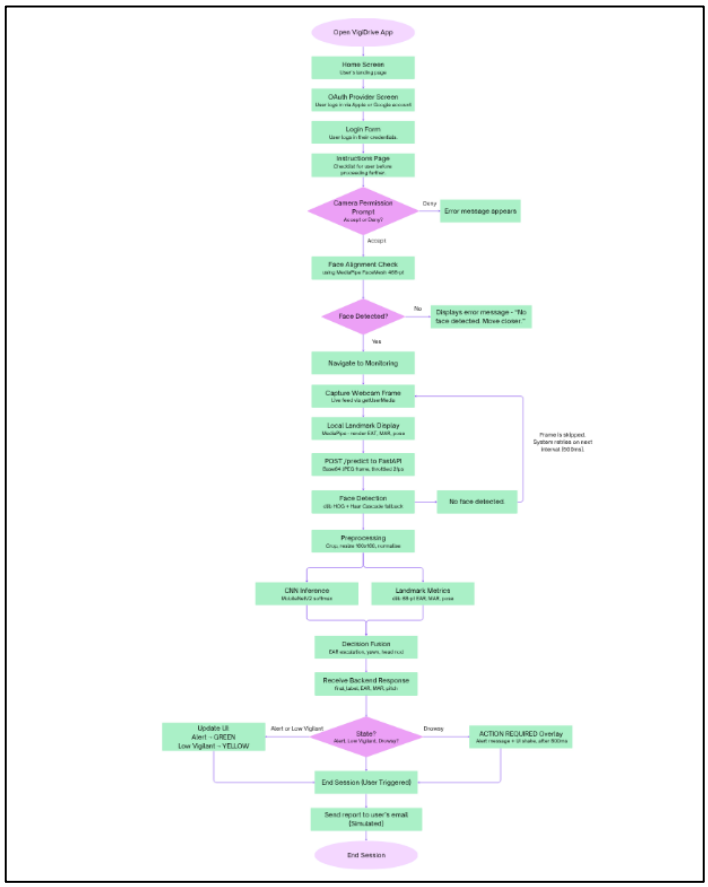


Figure 3.1.1: Flowchart of our application, VigiDrive.

3.2 CNN Architecture Design

Three CNN architectures were evaluated for the drowsiness classification task - MobileNetV2, EfficientNetB0, and ResNet50V2, each pre-trained on ImageNet and adapted using a two-phase transfer learning strategy. The architectures were compared under Leave-One-Subject-Out cross-validation to identify the model that generalises most reliably to unseen drivers, with particular emphasis on correctly detecting the Drowsy class as the safety-critical outcome.

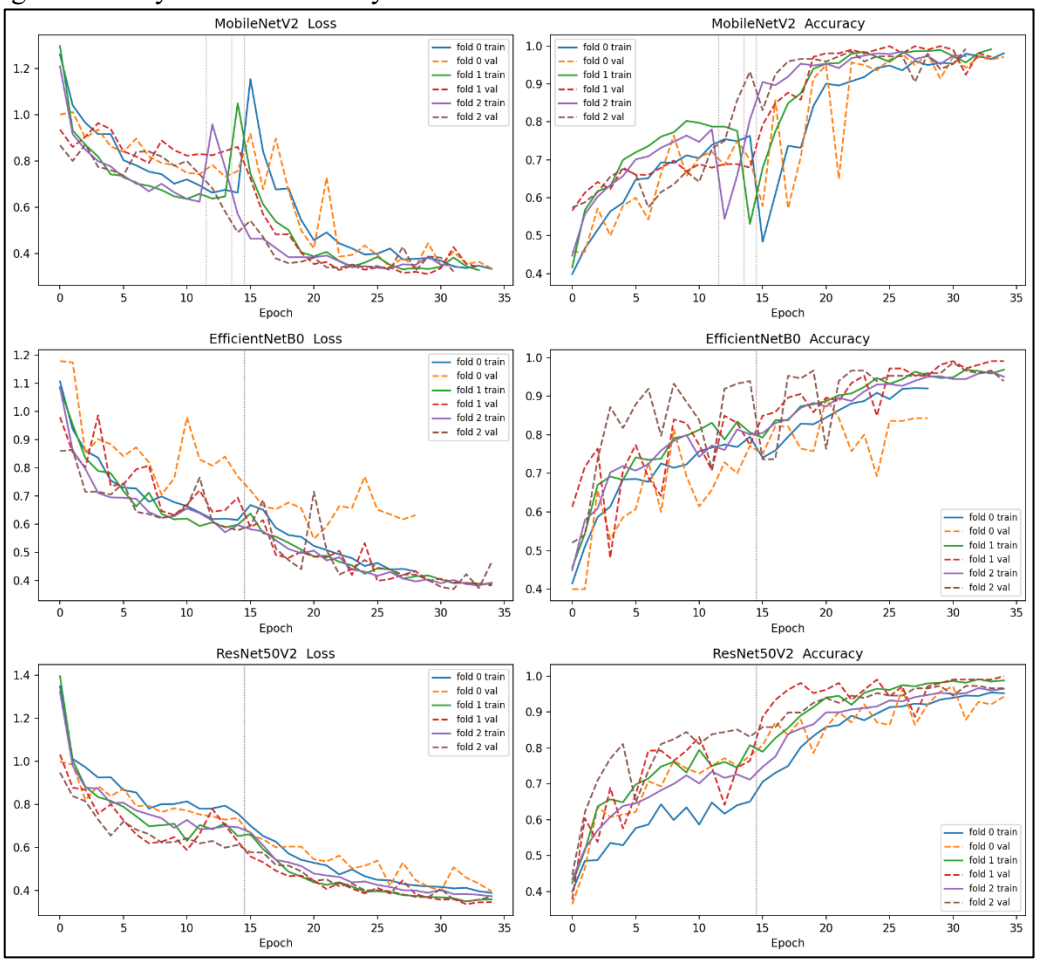


Figure 3.2.1: Training and Validation Loss, Accuracy Curves.

Figure 3.2.1 displays the loss and accuracy across all three LOSO folds for MobileNetV2, EfficientNetB0, and ResNet50V2. The vertical dotted line marks the Phase 1 to Phase 2 transition where the backbone is partially unfrozen. MobileNetV2 shows the most stable convergence - loss decreases smoothly across all folds and validation accuracy climbs steadily into the 0.90 - 0.95 range by the end of training. EfficientNetB0 exhibits higher variance in the validation curves, particularly in early Phase 2, likely due to its internal rescaling layer interacting with the fine-tuning schedule. ResNet50V2 converges more slowly in Phase 1 but catches up strongly in Phase 2, with all three folds reaching above 0.90 validation accuracy. Across all three architectures, the phase transition visibly accelerates learning, confirming that fine-tuning the top backbone layers contributed meaningfully beyond head-only training.

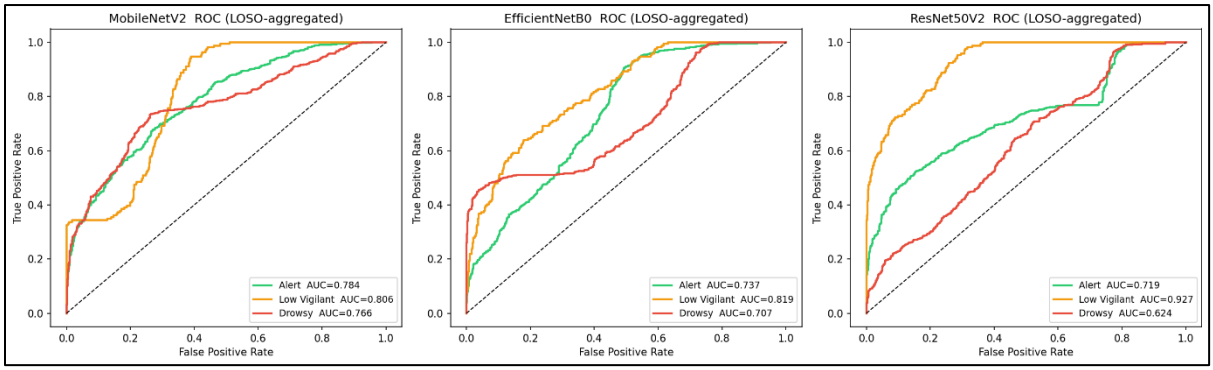


Figure 3.2.2: ROC Curves per Class, LOSO-Aggregated, All 3 Architectures.

Figure 3.2.2 shows the ROC curves comparing the one-vs-rest discriminative ability of each architecture across the three vigilance classes. MobileNetV2 achieves balanced performance with AUC values of 0.784 for Alert, 0.806 for Low Vigilant, and 0.766 for Drowsy — the most consistent spread across all three classes. EfficientNetB0 achieves a strong Low Vigilant AUC of 0.819 but its Drowsy AUC drops to 0.707, which is a concern for a safety-critical system where missed drowsiness events carry the highest risk. ResNet50V2 produces a notably high Low Vigilant AUC of 0.927 but severely underperforms on Drowsy at 0.624, making it the least suitable architecture for this task despite strong aggregate numbers. MobileNetV2's more balanced class-level AUC profile, combined with its inference speed advantage, confirmed it as the final chosen model.

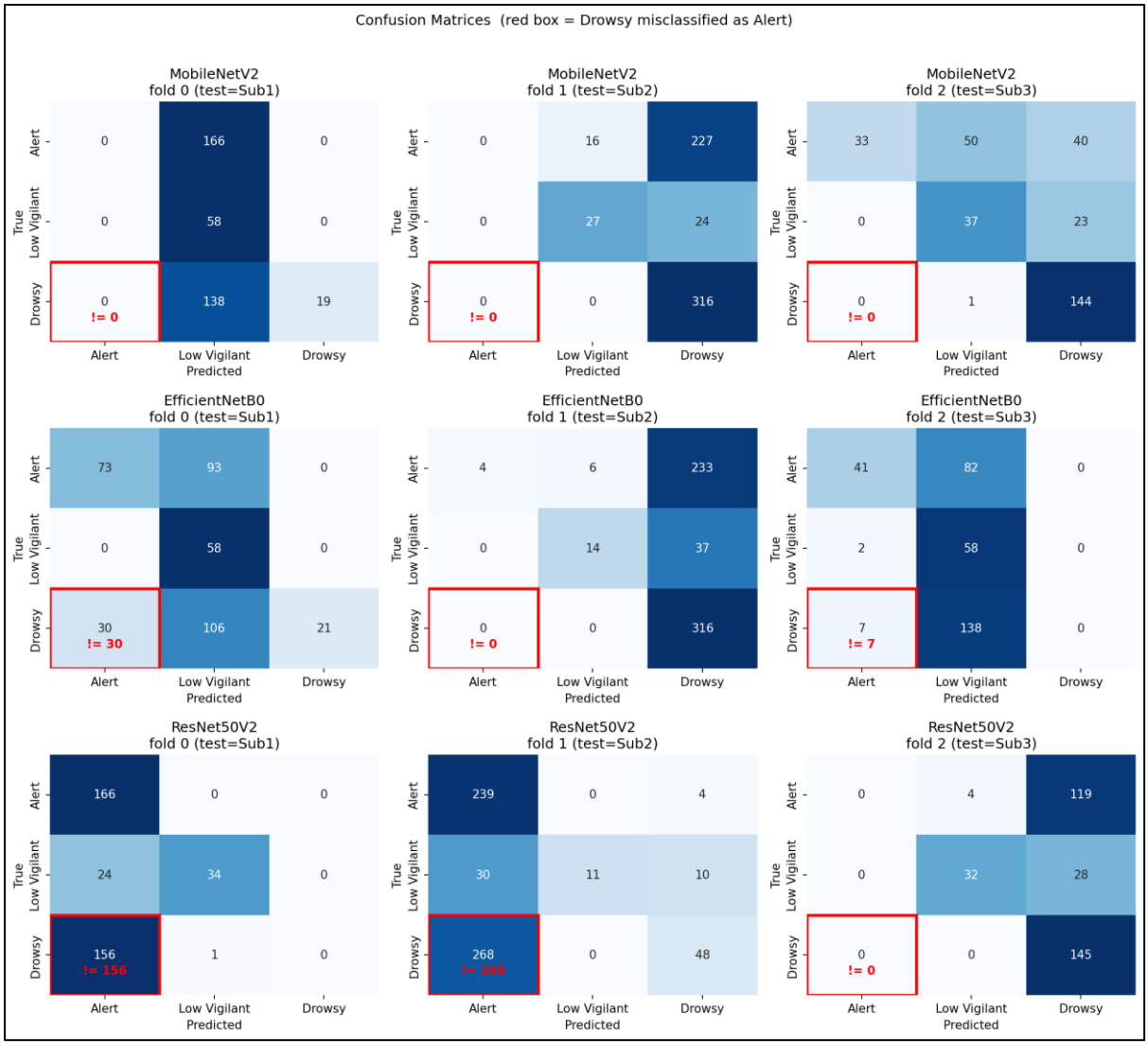


Figure 3.2.3: Confusion Matrices per Architecture per Fold (LOSO, red box = Drowsy misclassified as Alert).

The confusion matrices in Figure 3.2.3 reveal per-subject generalisation behaviour across all nine training runs. The red box highlights the most safety-critical error - Drowsy samples misclassified as Alert, as this represents a failure that would not alert the driver. MobileNetV2 records zero Drowsy-as-Alert misclassifications across all three folds, which is the strongest result across all architectures. EfficientNetB0 misclassifies 30 Drowsy samples as Alert in Fold 0 and 7 in Fold 2, indicating subject-dependent vulnerability. ResNet50V2 performs the worst on this metric, misclassifying 156 Drowsy samples as Alert in Fold 0 and 268 in Fold 1, making it unsuitable for deployment despite strong validation accuracy. MobileNetV2's perfect Drowsy-Alert separation across all subjects was the deciding factor in selecting it as the production model for the VigiDrive backend.

3.3 GUI Design

The frontend of our app, VigiDrive, was developed using React instead of Streamlit, even though the backend is entirely Python-based. Although Streamlit is effective for rapidly prototyping machine learning dashboards, it is not suitable for a high-traffic, real-time application like VigiDrive due to its limited customization capabilities and architecture.

Thus, React was selected because it offers complete control over the user interface, animations, and interactivity while efficiently updating only the components that change. For example, when the

driver's state changes from Alert to Drowsy, only the relevant UI elements such as the status label, border colour and statistics panel are re-rendered, while the remainder of the interface remains unaffected. This ensures a smooth, responsive and real-time monitoring experience.

React enables smooth multi-screen navigation, including Login, Instruction, Monitoring and Session Summary pages, with seamless animated transitions that would be difficult to implement efficiently in Streamlit. The system follows a clear separation of concerns, where React handles the interactive frontend and the Python backend manages tasks such as MobileNetV2 inference, EAR and MAR computation using MediaPipe and email report generation. Both components communicate through a lightweight API, allowing independent development, testing, and scalability.

Elements	Description
BG	Fixed-position animated background: radial gradient glows (teal & blue corners) and 20 randomly placed blinking star dots.
Logo	Renders the VigiDrive .png at a configurable size. Accepts a size prop (default 40px) so each screen can scale it independently.
Btn	Unified button supporting five variants: primary (purple fill), secondary (transparent border), danger (red fill), ghost (purple border), back (purple fill), and deny (red border).
Input	Labelled text/password field with a purple highlight border on focus.
Card	Frosted-glass container (backdrop blur + semi-transparent fill) for grouping content.
Divider	Horizontal rule with optional centred label text.

Table 2.3.1: Table of Shared Component Library.

Table 2.3.1 consists of a collection of pre-built, reusable UI elements, including buttons, input fields, cards and the animated background. They are defined once and referenced consistently across all screens of the application. This approach eliminates redundant styling, ensures visual uniformity throughout the interface and simplifies future modifications by centralising design changes to a single location.

Element	Description
fadeUp	Elements slide up and fade in when a screen first mounts (staggered with .fade-up-1 to .fade-up-5 classes).
float	The logo gently bobs vertically (4 s ease-in-out infinite) on home, OAuth, and login screens.
pulseRing	Teal glow ring that pulses around the user avatar during a live monitoring session.
scan	Horizontal purple line sweeping across the camera preview on the permissions screen.
blink	Individual star dots fade in and out at randomised speeds on the BG component.
shake	The status label shakes horizontally when the driver state is DROWSY.
heatPulse	The Grad-CAM heatmap overlay pulses opacity to indicate it is live.
gradientShift	Animated gradient background shift (defined but reserved for future use).

Table 2.3.2: Table of Global Animations.

Table 2.3.2 are a set of CSS keyframe sequences defined once at the application level and applied selectively across screens to enhance the user experience. They govern motion behaviours such as screen transition effects, the floating logo, the live camera scanning line, and the drowsiness alert shake effect. By declaring all animations centrally, the application maintains consistent motion design without duplicating animation logic across individual components.

3.4 Screen Flow

3.4.1 Home Page

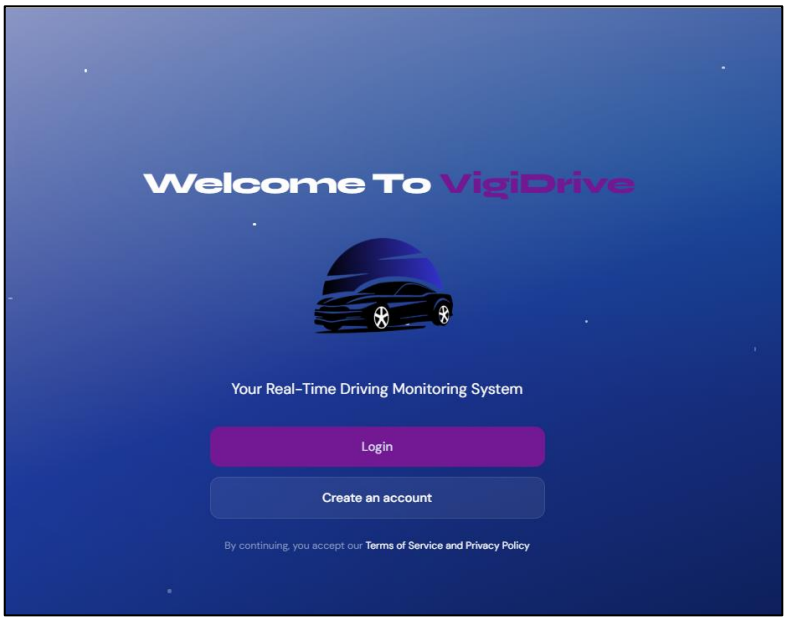


Figure 3.4.1.1: Screenshot of Home or Landing Page.

Figure 3.4.1.1 shows the Home Page, the first page that the user sees when they open the application. The animated background runs continuously behind all content. Centred on the screen is the heading *"Welcome To VigiDrive"* with the brand name highlighted in purple, followed by the floating app logo, and a subtitle reading *'Your Real-Time Driving Monitoring System'*. Below these, two buttons are presented, a solid purple Login button and a secondary Create an Account button. Clicking Login takes the user forward to the OAuth screen. Clicking Create an Account opens Google's sign-up page in a new browser tab, since account registration is handled externally through Google. User's should already have an email account in order to proceed using the application.

3.4.2 OAuth Login Page

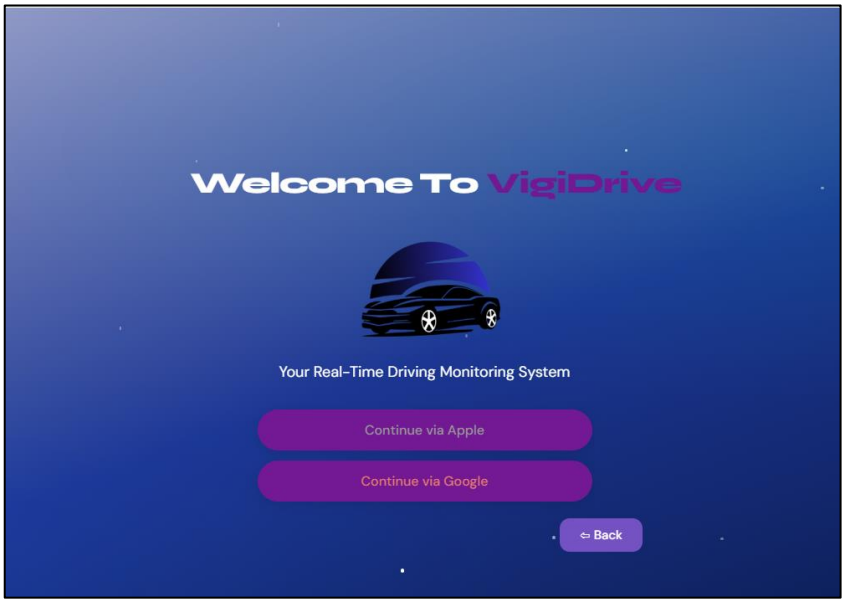


Figure 3.4.2.1: Screenshot of OAuth Login Page.

Figure 3.4.2.1 shows the OAuth Login Page screen that maintains the same visual layout as the Home Page. There are two primary buttons, 'Continue via Apple' and 'Continue via Google'. Both buttons lift and shift colour slightly on hover to provide tactile feedback. **For demonstration purposes, both buttons route to the same email and password login form**. The purpose of this screen is to demonstrate the provider selection interface rather than a live OAuth integration. A Back button in the lower right returns the user to the Home Page.

3.4.3 Login Page

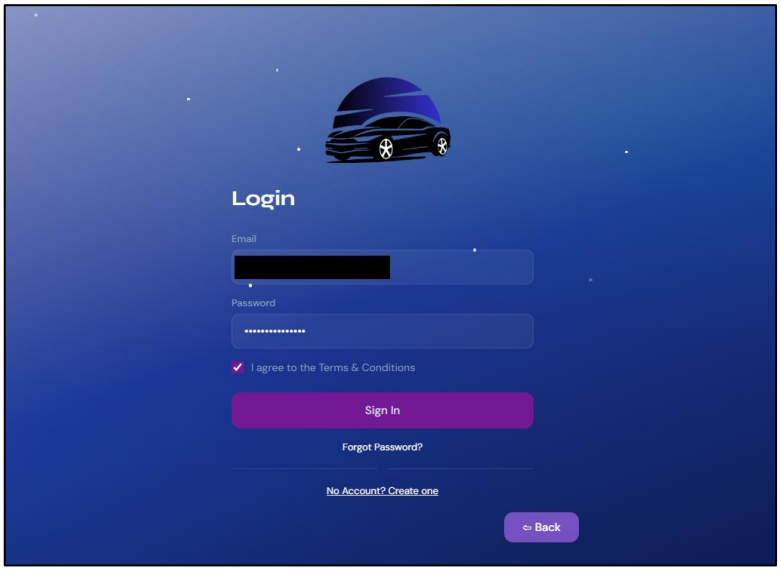


Figure 3.4.3.1: Screenshot of Login Page.

Figure 3.4.3.1 presents a standard credential login form. The application logo sits above a Login heading, followed by an Email field and a Password field. Both fields are pre-filled with a dummy test account for convenience during assessment and each field's border turns purple when the user clicks into it. Below the fields is a *"Terms and Conditions"* checkbox, which must remain ticked for the Sign

In button to be active. If any of the 3 fields are unchecked, the button is greyed out and non-interactive. Once the user clicks Sign In, they are taken to the Instructions Page. A Forgot Password link opens Google's account recovery page in a new tab and a *"No Account? Create one"* link opens Google's registration page.

3.4.3.1 Google Authentication via OAuth 2.0 Protocol

The authentication system follows the OAuth 2.0 protocol using Google and Apple as identity providers. When a user selects *"Continue with Google"*, they are redirected to Google's authorisation server. Upon granting consent, Google issues an identity token containing the user's verified email address and unique Google ID. VigiDrive's backend receives this token, verifies its signature against Google's public keys and checks whether the email already exists in the VigiDrive user database. If the user is new, a record is automatically created containing only their Google ID, email address and registration timestamp, with no password stored by VigiDrive at any point. If the user already exists, they are logged in directly.

3.4.3.2 Apple Authentication via OAuth 2.0 Protocol

The same flow applies when a user selects *"Continue via Apple"*, Apple's authorisation server issues a verified identity token which VigiDrive validates and uses to look up or create a user record in the same way. In both cases, the respective identity provider notifies and synchronises with the VigiDrive database whenever a new account is created or updated, meaning VigiDrive always reflects the latest account state without managing credentials itself. Since Apple account creation is handled natively through a user's Apple device or existing Apple ID, no in-app registration link is provided for Apple. Users are expected to create or manage their Apple account externally before signing in. This approach delegates all credential management and password recovery to the respective provider, meaning *"Forgot Password"* redirects users to Google's own account recovery flow. The system consequently benefits from the robust security infrastructure already established by both providers, encompassing two-factor authentication, brute-force protection and suspicious login detection, eliminating the need for VigiDrive to develop or maintain these security mechanisms independently.

3.4.3.3 Scope Exclusions: Authentication Database

Since VigiDrive was developed as an academic prototype aimed primarily at demonstrating the core drowsiness detection functionality, implementing a complete authentication database was considered outside the scope of the project. The OAuth 2.0 integration with Google and Apple was therefore implemented at the conceptual and interface level only, while credential management, user account handling and data synchronisation were delegated to the respective identity providers. This approach enabled the development process to remain focused on the real-time driver monitoring and detection system, which represents the central objective of the project.

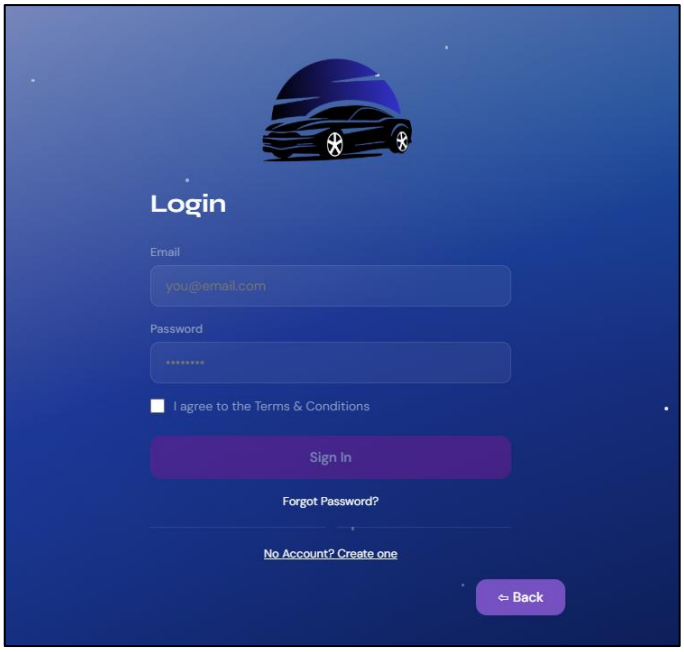


Figure 3.4.3.2: Screenshot of Login Page (Initial State).

Figure 3.4.3.2 shows the initial state of the Login page when it first loads. The Sign In button is disabled and greyed out because all the three fields are empty. The user must fill in their Email and Password and tick the checkbox, before the button becomes active and allows them to proceed. This is also the page that users will view to enter their credentials (without the dummy credentials set for demonstration purposes.)

3.4.4 Instructions Page

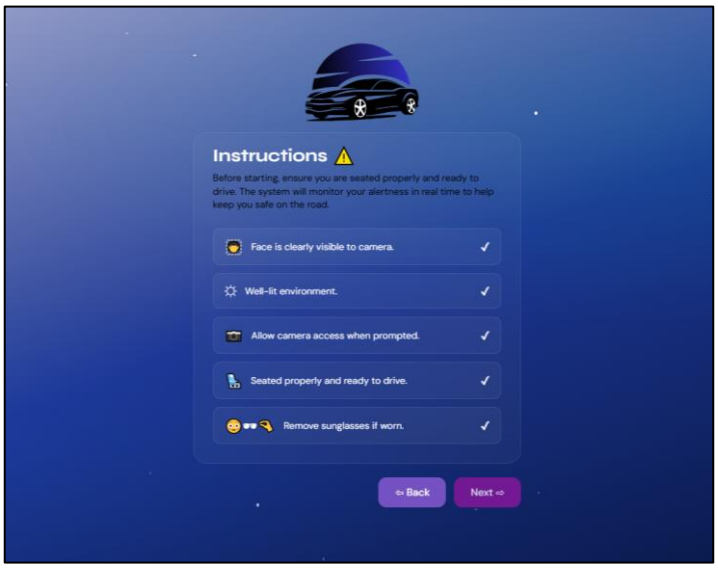


Figure 3.4.4.1: Screenshot of Instructions Page.

Figure 3.4.4.1 displays the Instructions Page after user has successfully logged into their account. Before the camera is activated, the page ensures that the driver is properly set up for accurate detection. A frosted-glass card presents five preparation requirements - face clearly visible to the camera, a well-lit environment, camera access allowed when prompted, seated properly and ready to drive and

sunglasses removed if worn. Each item is displayed as a small row with an emoji icon on the left and a tick mark on the right, similar to a checklist. After reviewing the instructions, the driver clicks *"Next"* to proceed to the camera permission screen or *"Back"* to return to the Login page.

3.4.5 Camera Permission and Face Detection Page

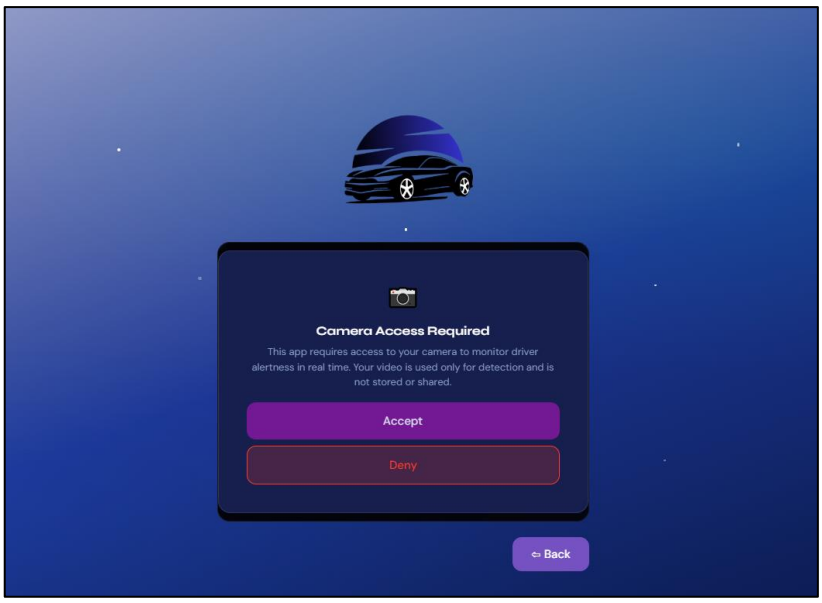


Figure 3.4.5.1: Screenshot of Camera Permissions Page.

Figure 3.4.5.1 screen manages two sequential steps - requesting camera access from the browser and then verifying that the driver's face is correctly positioned before monitoring begins. When the user clicks *"Accept"*, the application requests access to the device camera so the real-time monitoring process can begin. Once permission is granted, VigiDrive activates the webcam and starts analysing the driver's facial features, including eye movement, blinking patterns, mouth opening and head posture. These visual inputs are processed using the trained MobileNetV2 model together with facial landmark analysis to detect signs of low vigilance or drowsiness. The captured video is used only for live monitoring during the session and is not permanently stored or shared.

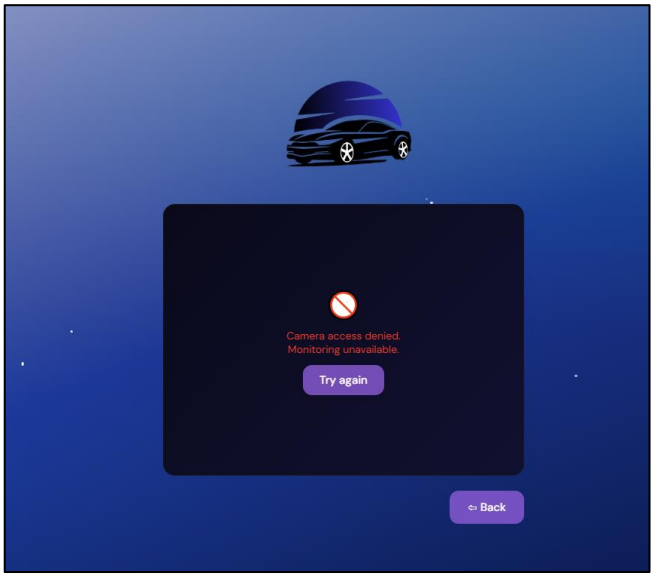


Figure 3.4.5.2: Camera access denied state.

Figure 3.4.5.2 shows that when the user clicks *"Deny"*, camera access is rejected and the monitoring system cannot be activated. Since VigiDrive relies on real-time facial analysis for drowsiness detection, denying permission prevents the application from tracking eye closure, yawning and head movement. As a result, the user cannot proceed to the driver monitoring stage until camera access is granted. This ensures that the system operates only with explicit user consent and maintains user privacy and control over device permissions.

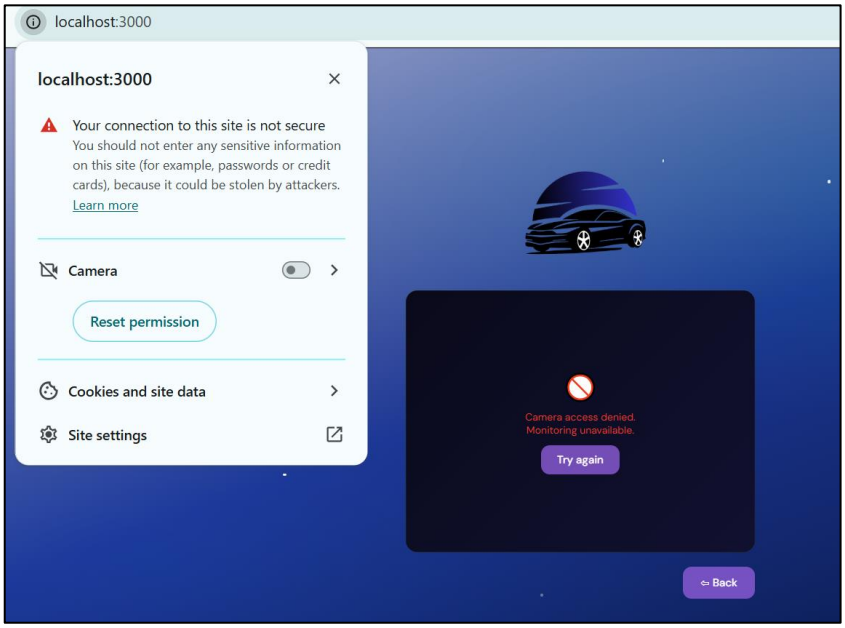


Figure 3.4.5.3: Camera access is disabled when clicked on "Accept".

Figure 3.4.5.3 shows that when the user clicks *"Accept"* but their browser camera is turned off, the page displays the same message shown after clicking the *"Deny"* button - *"Camera access denied. Monitoring unavailable."*

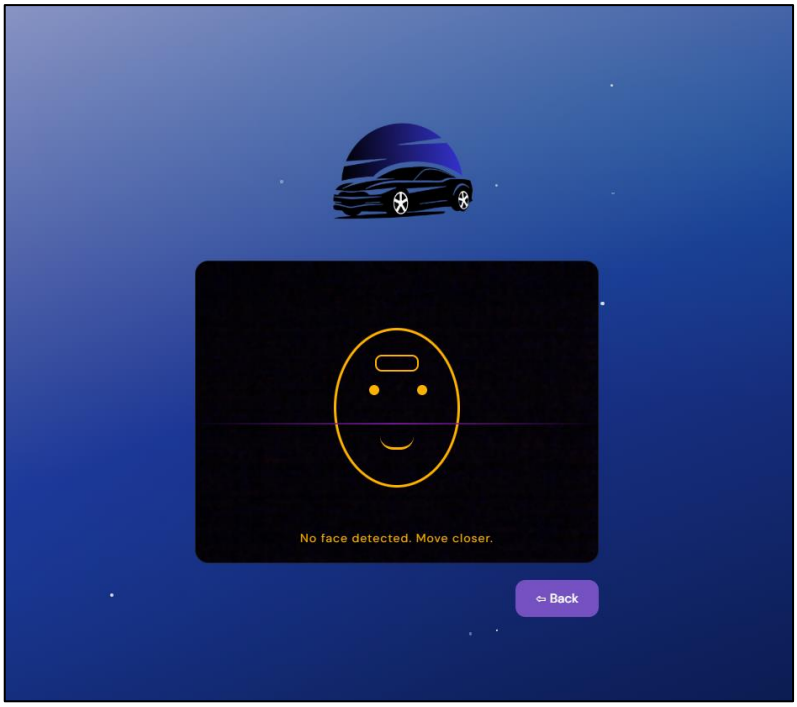


Figure 3.4.5.4: Screenshot displaying the closed camera shutter state.

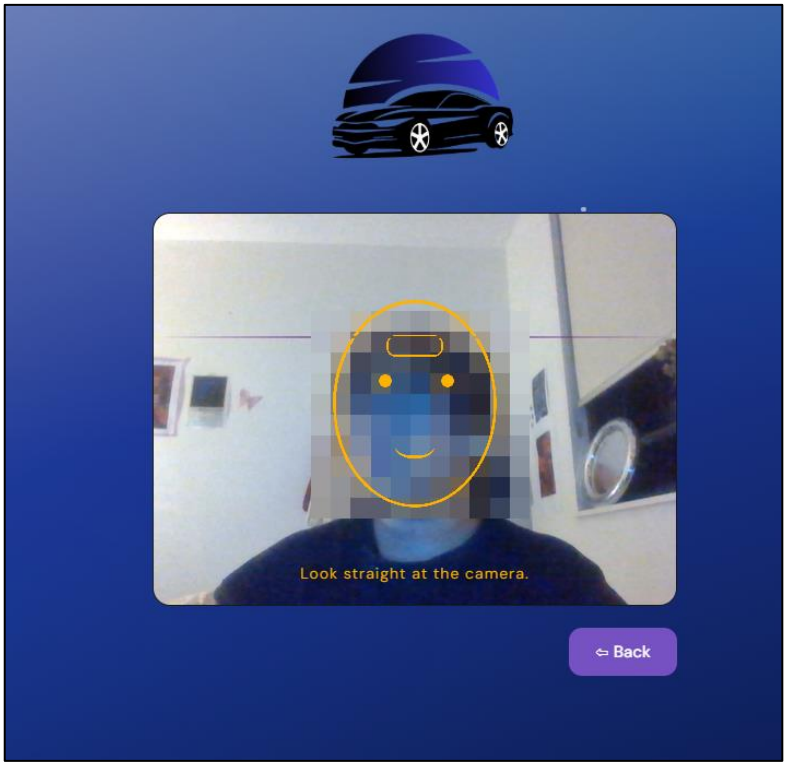


Figure 3.4.5.5: Active camera feed showing the user looking to the side.

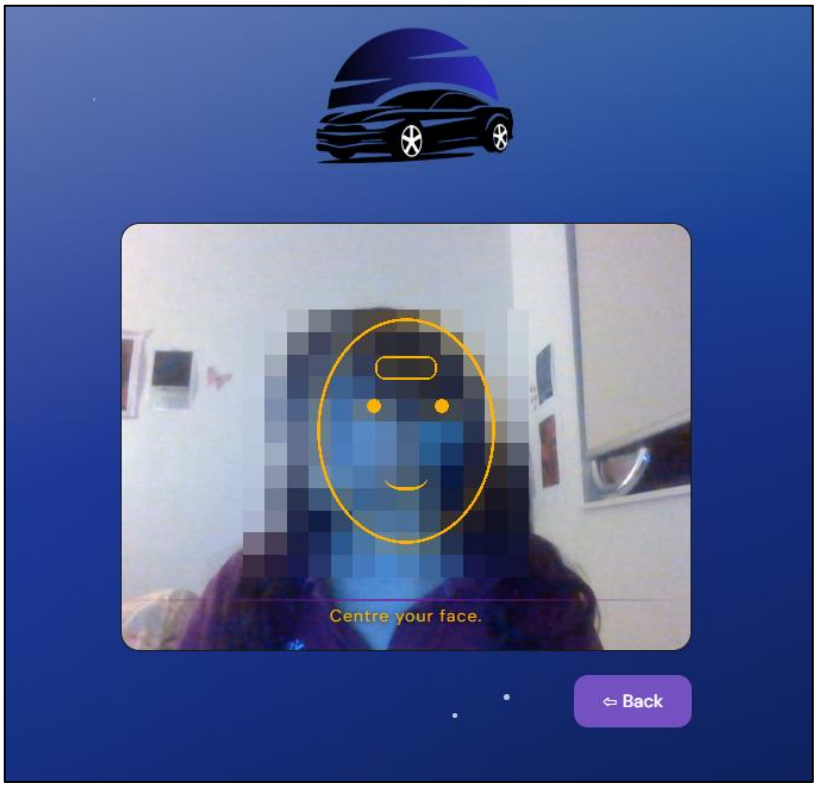


Figure 3.4.5.6: Camera is active, and the user's gaze is not directed at the camera.

Based on Figure 3.4.5.4, Figure 3.4.5.5 and Figure 3.4.5.6, while the alignment check runs, a face outline ellipse, two eye position markers, a mouth arc and a forehead indicator are drawn over the video feed in orange. A dynamic status message below the overlay tells the user what adjustment is needed.

For example, "*Centre your face", "Move closer to the camera",* or *"Look straight at the camera".* The system checks six conditions on each frame simultaneously - the nose tip must be horizontally centred, both the forehead and mouth must be within the frame boundaries, the face must not be turned left or right (yaw check), the head must not be tilted up or down (pitch check), the head must not be rolled sideways (roll check) and the face must be at the correct distance from the camera.

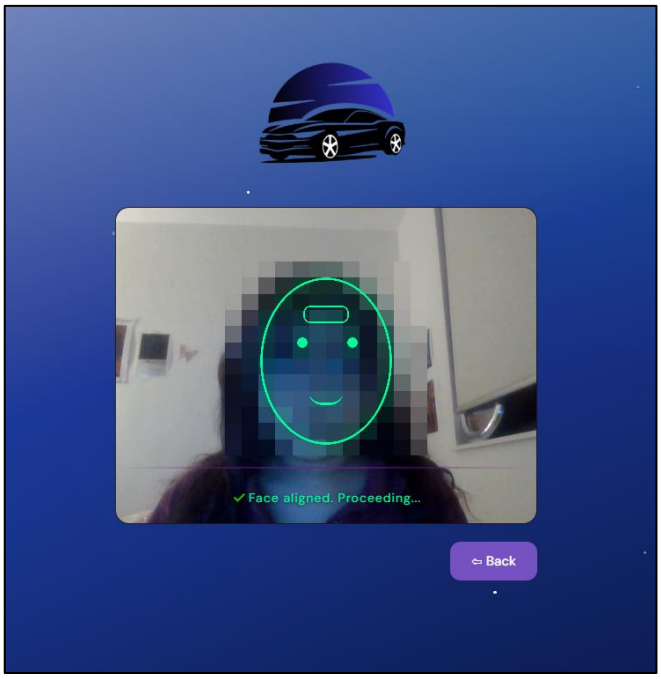


Figure 3.4.5.7: User face aligned and looking directly into the camera.

Figure 3.4.5.7 shows that once all six conditions pass at the same time, the overlay elements turn green, the message changes to *"Face aligned. Proceeding..."* and the app automatically navigates to the monitoring screen after a 1.5 second pause.

3.4.6 Live Monitoring Page

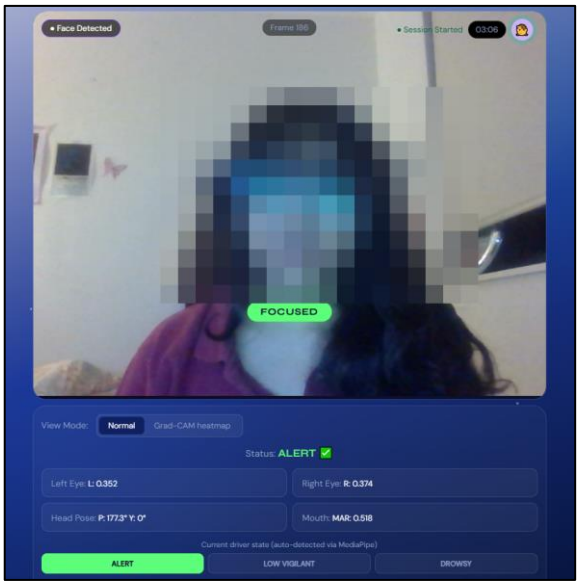


Figure 3.4.6.1: Live Monitoring Screen, Alert State (Normal View).

Figure 3.4.6.1 shows the monitoring screen in which the driver/user is in an Alert state, with a status label reading “FOCUSED” and the ALERT indicator highlighted green in the state panel. The normal view mode is active, showing facial landmark tracking overlaid on the live webcam feed. Real-time metrics confirm the driver's eyes are open with Left EAR: 0.352 and Right EAR: 0.374 and the session timer reads 03:06.

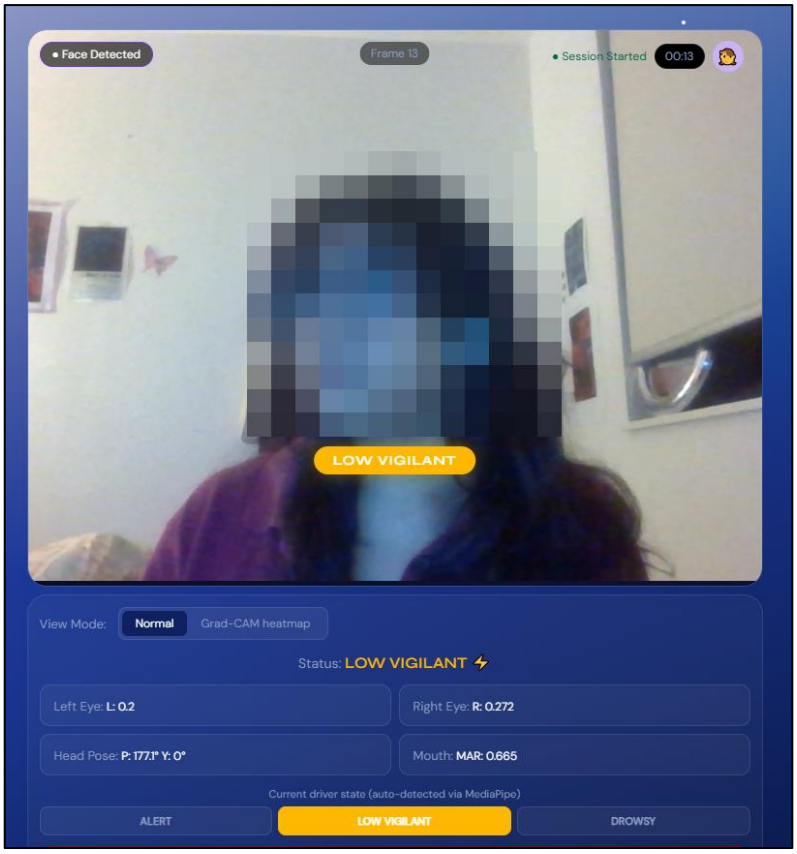


Figure 3.4.6.2: Live Monitoring Screen, Low Vigilant State (Normal View).

Figure 3.4.6.2 shows the monitoring screen that detects early fatigue signs and transitions to a Low Vigilant state, with the status label displaying “LOW VIGILANT” in amber and the LOW VIGILANT indicator highlighted in the state panel. The EAR values have dropped to Left: 0.264 and Right: 0.25, indicating partial eye closure, while the MAR reading of 0.833 suggests the driver is yawning. The session timer reads 00:53.

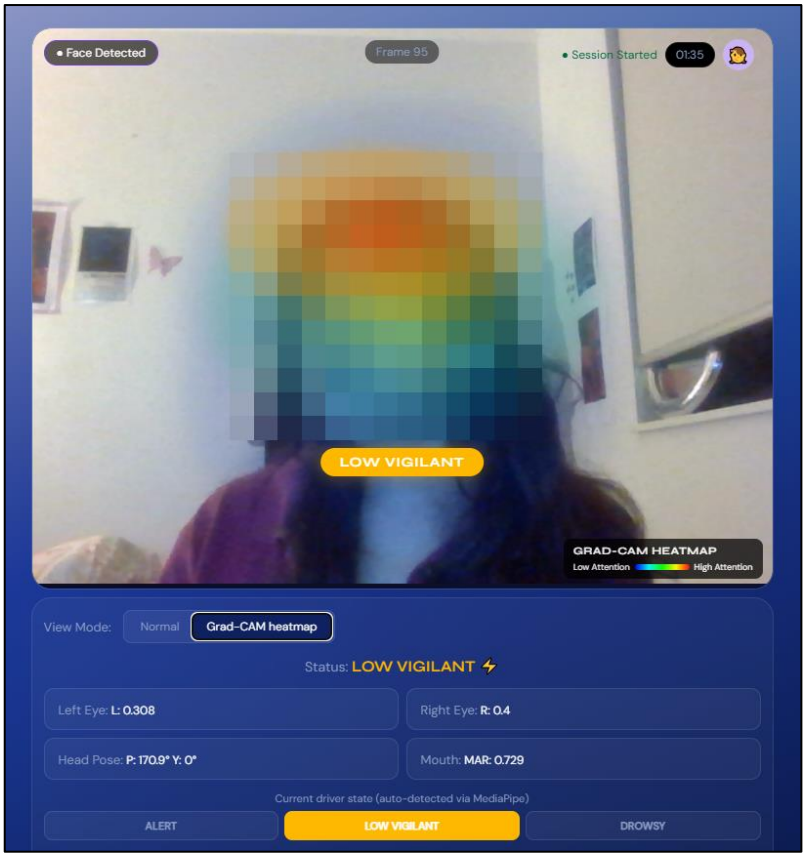


Figure 3.4.6.3: Live Monitoring Screen, Low Vigilant State (Grad-CAM Heatmap View).

Figure 3.4.6.3 shows the driver's Eye Aspect Ratio dropping below the Low Vigilant threshold, the system transitions from Alert to Low Vigilant state, updating the status label to "LOW VIGILANT" in amber and highlighting the corresponding state indicator in the panel below. Simultaneously, the Grad-CAM heatmap overlay activates on the camera feed, visualising the regions the MobileNetV2 model focused on when making its classification, with warmer colours indicating higher attention, concentrated around the eye and forehead region where drowsiness indicators such as partial eye closure are most prominent. The real-time metrics confirm the detection, with the Left EAR at 0.308 and Right EAR at 0.4 indicating partially closed eyes and a MAR of 0.729 suggesting early yawning activity, both of which contributed to the Low Vigilant classification.

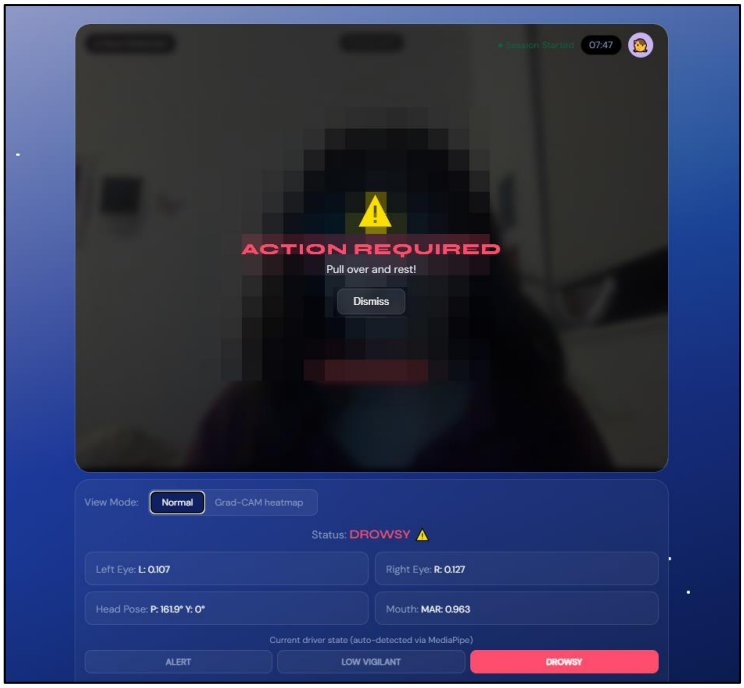


Figure 3.4.6.4: Drowsy State with ACTION REQUIRED Alert.

Figure 3.4.6.4 illustrates the monitoring screen that triggers a full-screen ACTION REQUIRED overlay after sustained drowsiness is detected, with the status reading "DROWSY" and the DROWSY indicator highlighted in red. The EAR values have critically dropped to Left: 0.107 and Right: 0.127, confirming prolonged eye closure. A warning overlay prompts the driver to *"Pull over and rest!"* with a *"Dismiss"* button to temporarily clear the alert. The session timer reads 07:47

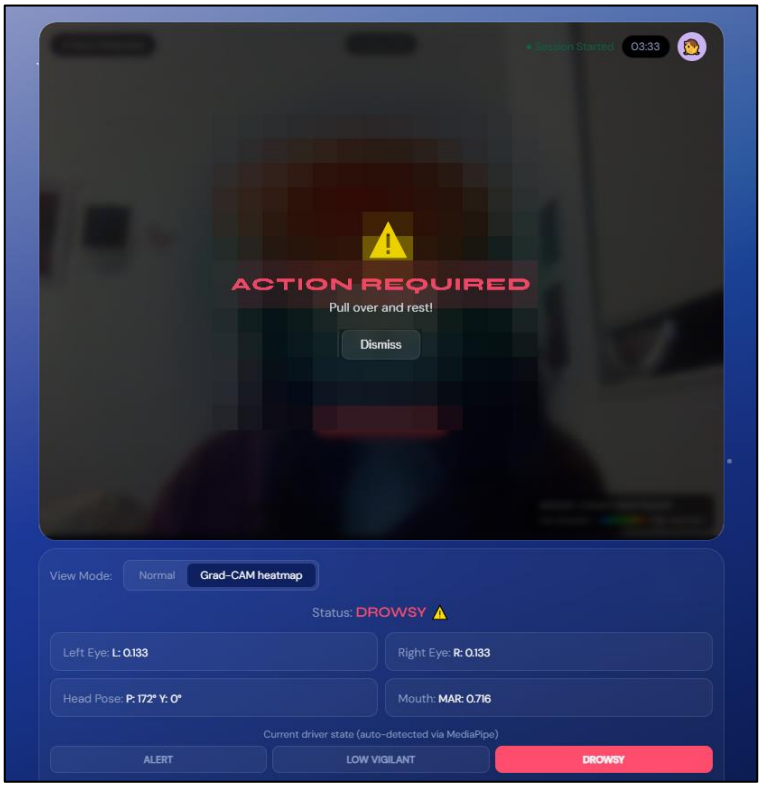


Figure 3.4.6.5: Live Monitoring Screen, Drowsy State with ACTION REQUIRED Alert (Grad-CAM Heatmap View)

Based on Figure 3.4.6.5, when the driver's EAR drops critically below the Drowsy threshold, the system escalates from Low Vigilant to Drowsy, triggering a full-screen ACTION REQUIRED overlay with a warning icon and the instruction *"Pull over and rest!"* accompanied by a Dismiss button. The status panel confirms the Drowsy classification with the DROWSY indicator highlighted in red, and the real-time metrics show critically low EAR values of Left: 0.133 and Right: 0.133, indicating sustained eye closure that crossed the detection threshold. The Grad-CAM heatmap view remains active in the background, continuing to visualise the model's attention regions on the driver's face, while the session timer at 03:33 indicates the drowsiness event occurred within the first few minutes of the monitoring session.

3.4.7 Session Ending Page

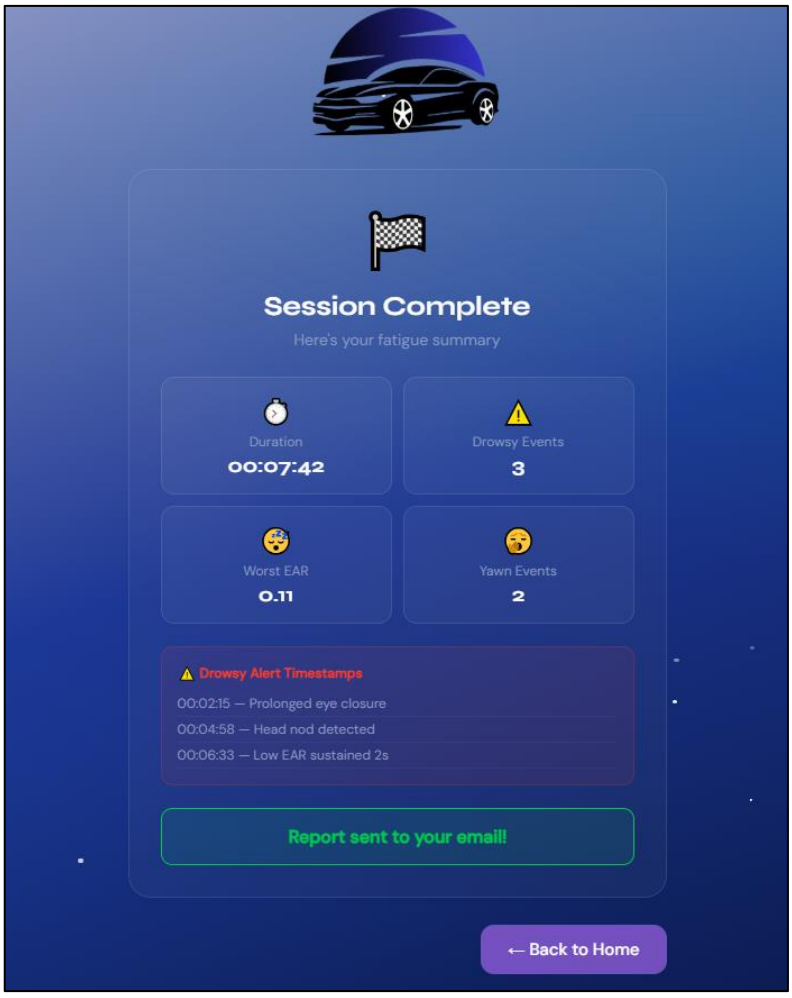


Figure 3.4.7.1: Session End Page, Fatigue Summary and Email Report.

Figure 3.4.7.1 displays the email report functionality which is implemented at the interface level, where the session summary, including total duration, drowsy event count, worst EAR recorded, and timestamped alert log is displayed on the Session End screen with an *"Email session report"* button. In the current prototype, clicking the button simulates the send request with a 1.8 second delay before displaying a confirmation message, as full email delivery requires a backend mail service such as Python's smtplib or a third-party provider like SendGrid to be configured with authenticated SMTP credentials. This integration was intentionally deferred as it falls outside the core scope of the drowsiness detection system, which focuses on real-time facial analysis and classification. In a

production deployment, the backend would collect session metrics throughout the monitoring session, construct a formatted report, and dispatch it to the user's registered email address upon session completion.

3.5 Backend Implementation

The VigiDrive backend is a RESTful API built with FastAPI and hosted on Google Colab during development and demonstration. It serves as the inference engine for the drowsiness detection system, receiving webcam frames from the React frontend, running the full detection pipeline, and returning classification results in real time. The backend is exposed publicly via ngrok, which creates a secure HTTPS tunnel from the Colab runtime to the internet, allowing the locally running React application to communicate with it across the network.

3.5.1 Technology Stack

The backend is written in Python and uses the following core libraries. FastAPI serves as the web framework, providing automatic request validation, async request handling, and an auto-generated interactive documentation page at /docs. Uvicorn is the ASGI server that hosts the FastAPI application. TensorFlow and Keras are used to load and run the pre-trained MobileNetV2 classification model. dlib provides accurate frontal face detection and 68-point facial landmark prediction, which are used to compute the Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and head pitch angle on every frame. OpenCV handles image decoding, colour space conversion, face cropping, and head pose estimation via solvePnP. Pydantic validates all incoming request data, and CORSMiddleware allows the React frontend running on localhost to make cross-origin requests to the ngrok URL without being blocked by the browser.

3.5.2 Deployment in Google Colab

The backend was deployed in Google Colab rather than locally because the required packages, particularly dlib and TensorFlow, could not be installed successfully on the local Windows machine due to missing build tools and dependency conflicts. Colab provides a pre-configured Linux environment where all packages install cleanly and the trained model file is accessible via Google Drive. The Colab notebook handles the full startup sequence. First, Google Drive is mounted and the working directory is set to the vigidrive project folder containing backend.py, the model file and the dlib shape predictor. A compatibility patch is then applied to BatchNormalization to strip unrecognised parameters that were saved with an older version of Keras, preventing a TypeError when the model loads. nest_asyncio is applied to allow uvicorn to run inside Colab's existing event loop, and any existing ngrok tunnels are killed before a new HTTPS tunnel is opened on port 8000. The FastAPI server is started in a background daemon thread, so the notebook remains interactive while the server runs continuously.

A static ngrok domain is permanently assigned to this account, so the *BACKEND_URL* in App.jsx never needs to be updated between sessions.

3.5.3 Detection Pipeline

Every 500 milliseconds, the React frontend captures a frame from the webcam, encodes it as a base64 JPEG string and sends it to the POST or predict endpoint. The backend processes each frame through a 6-step pipeline.

1. **Image Decoding**: The base64 string is decoded into raw bytes and converted into an OpenCV BGR image array.

2. **Face Detection:** dlib's HOG face detector locates the driver's face. If dlib finds no face, OpenCV's Haar Cascade is used as a fallback.

3. **Landmark Extraction:** The dlib 68-point shape predictor locates key facial landmarks. EAR is computed separately for each eye using the vertical-to-horizontal distance ratio of the eyelid points. MAR is computed from the outer lip boundary points. Head pitch is estimated using OpenCV's solvePnP with a generic 3D face model.

4. **CNN Inference:** The detected face is cropped, resized to 160x160 pixels, converted to RGB, and preprocessed for MobileNetV2. The model outputs three class probabilities: Alert, Low Vigilant, and Drowsy.

5. **Decision Fusion**: The final classification is determined by direct EAR and MAR thresholds rather than the CNN output alone. If the average EAR falls below 0.20 the driver is classified as Drowsy; if EAR is below 0.27 or MAR exceeds 0.55, the driver is classified as Low Vigilant. This override was introduced because the CNN was trained on a specific in-car dataset and did not generalise reliably to all webcam environments. The DecisionFusion class also tracks consecutive frame streaks to prevent single-frame false positives and fires the alert trigger after sustained drowsiness is confirmed.

6. **Response:** The final label, CNN confidence score, left and right EAR, MAR, head pitch, and alert flags are returned as a JSON object to the frontend, which updates the status display and metric panels in real time.

3.5.4 Limitations

Since the backend runs on Google Colab and communicates with the frontend via ngrok, each frame incurs a network round-trip latency of approximately 500 to 1000 milliseconds. This means the displayed metrics and status update with a slight delay relative to the driver's actual movements. In a production deployment, the backend would run locally on the same machine as the frontend, eliminating this latency entirely and enabling near real-time response. Additionally, the email report functionality is implemented at the interface level only; full email delivery would require a configured mail service such as Python's smtplib or a provider like SendGrid, which was deferred as it falls outside the core scope of the project.

4. Results and Evaluation

This section reports the outcomes of training three CNN backbones on the D3S annotated dataset under a subject-disjoint leave-one-subject-out (LOSO) protocol, alongside the decision-fusion module and end-to-end latency benchmarks. The backbones evaluated were MobileNetV2 (Sandler et al., 2018), EfficientNetB0 (Tan & Le, 2019), and ResNet50V2 (He et al., 2016).

4.1 Experimental Settings

D3S contains video frames of three participants (Sub1, Sub2, Sub3) performing four expressions: Neutral, Happy, Yawn, and Eye-close. Sub1 does not include the Happy state. Following the protocol justified in Section 3, the four expressions were mapped to a 3-class drowsiness taxonomy (Neutral + Happy to Alert, Yawn to Low Vigilant, Eye-close to Drowsy). The mapping yielded 532 Alert frames, 169 Low Vigilant frames, and 618 Drowsy frames after face-detection filtering, with per-subject counts of 378 / 605 / 324 for Sub1 / Sub2 / Sub3 respectively.

LOSO cross-validation was used to expose the real generalisation behaviour of each model: in each of three folds, two subjects formed the training pool and the third was held out as the test set. Within the training pool, 15% of frames were randomly sampled (stratified by class) as a validation set for EarlyStopping and ReduceLROnPlateau. Fold splits are as follows in Table 4.1.1.

Fold	Test subject	Train frames	Val frames	Test frames
0	Sub1	798	140	381
1	Sub2	603	106	610
2	Sub3	843	148	328

Table 4.1.1: Subject-disjoint LOSO fold composition and per-fold frame counts.

To compensate for class imbalance, per-fold class weights were computed using sklearn.utils.class_weight.compute_class_weight('balanced'). The Low Vigilant class received the highest weight in every fold (between 1.91 and 3.02) because Yawn frames are the rarest expression in D3S.

All three backbones were trained using identical hyper-parameters except where the architecture made that impossible. Both training phases used the Adam optimiser with CategoricalCrossentropy(label_smoothing=0.1). Phase 1 froze the backbone (base.trainable = False) and trained only the classification head at LR 1e-3 for up to 15 epochs. Phase 2 unfroze a model-specific top portion of the backbone and continued at a lower learning rate for up to 20 more epochs. The exact configurations were:

Architecture	Backbone params	Unfreeze region	Phase 2 LR
MobileNetV2	2.26M	Layers 100-154 (55 layers)	1e-4
EfficientNetB0	4.05M	Top 20 layers, BN frozen	1e-4
ResNet50V2	23.6M	conv5_block onwards (~30 layers)	1e-5

Table 4.1.2: Per-architecture transfer-learning configuration for the three CNN backbones.

The classification head was identical across all three models: GlobalAveragePooling2D -> Dropout(0.3) → Dense(128, ReLU) → Dropout(0.2) → Dense(3, softmax). ReduceLROnPlateau (factor 0.2, patience 3 in Phase 1 and 5 in Phase 2, minimum LR 1e-7) and EarlyStopping (patience 6 / 8, restoring best weights) ran in both phases. ModelCheckpoint saved the lowest-val-loss model to disk.

Training was conducted on a Windows 10 machine with an Intel CPU and 16 GB RAM; the TensorFlow 2.21 build does not include native Windows GPU support, so all training and inference ran on CPU. Random seed 42 was set globally via tf.keras.utils.set_random_seed with op-level determinism enabled, so the three folds are reproducible.

The critical detail in our training pipeline is that the backbone was always called with training=False inside the functional API, even during Phase 2 fine-tuning. This keeps BatchNormalization layers in inference mode and prevents the small D3S batches from corrupting the ImageNet-pretrained BN moving statistics. This is the configuration recommended by the official Keras transfer-learning guide and is essential when fine-tuning ImageNet backbones on small datasets.

4.2 Pre-Processing

The pre-processing pipeline is documented in detail in Section 3. Briefly: dlib's HOG face detector locates each face with 20% padding around the bounding box, the crop is resized to 160 x 160 pixels, and converted to RGB. Per-architecture preprocess_input functions are applied inside the tf.data pipeline (MobileNetV2 and ResNet50V2 scale to [-1, +1]; EfficientNetB0 is a pass-through because the model contains an internal Rescaling layer). Augmentation is applied to training batches only and consists of RandomFlip("horizontal"), RandomRotation(±15°), RandomZoom(-0.1, +0.2), RandomBrightness(0.2), and RandomContrast(0.2). Vertical flips and any masking augmentation that could occlude the eye or mouth regions were explicitly excluded.

4.3 Experimental Results

4.3.1 LOSO Performance Comparison

Table 3 reports the headline numbers across the three architectures, averaged over the three LOSO folds.

Architecture	Accuracy	Macro F1	F1 (Drowsy)	Recall (Drowsy)	AUC (Drowsy)	Drowsy to Alert rate
MobileNetV2	0.472 ± 0.195	0.392 ± 0.172	0.583 ± 0.263	0.705 ± 0.413	0.901	**0.000**
EfficientNetB0	0.416 ± 0.101	0.344 ± 0.050	0.312 ± 0.291	0.378 ± 0.443	0.858	0.080
ResNet50V2	**0.518 ± 0.021**	0.437 ± 0.022	0.306 ± 0.273	0.384 ± 0.440	0.776	0.614

Table 4.3.1.1: Mean LOSO performance across architectures, mean ± std over the three folds.

The accuracy ranking does not match the safety ranking. ResNet50V2 has the highest mean accuracy at 0.518 and the lowest fold-to-fold variance, but it misclassifies 61.4% of Drowsy frames as Alert. MobileNetV2 has the lowest mean accuracy at 0.472 but never once mislabels a Drowsy frame as Alert across any of the three folds. For a drowsiness-detection system whose purpose is to wake a tired driver,

the Drowsy-to-Alert rate is the operationally important metric, and it does not correlate with overall accuracy.

The Drowsy class AUC reinforces this picture. MobileNetV2's AUC of 0.901 indicates that its softmax for the Drowsy class is well-calibrated as a ranking signal even when its argmax decisions miss; the model "knows" which frames are drowsy even when it does not commit to that label. ResNet50V2's AUC of 0.776 is meaningfully worse despite the higher accuracy, suggesting the larger model has learned something other than the underlying drowsiness signal on this dataset.

4.3.2 Fold-Level Breakdown

The mean numbers obscure substantial fold-to-fold variation. Table 4 shows MobileNetV2's per-fold results, which illustrate the heterogeneity that LOSO exposes.

Fold	Test subject	Accuracy	Macro F1	Drowsy F1	Drowsy to Alert
0	Sub1	0.549	0.532	0.825	0 / 157
1	Sub2	0.610	0.540	0.745	0 / 316
2	Sub3	0.256	0.105	0.180	0 / 145

Table 4.3.2.1: MobileNetV2per-fold performance under the LOSO protocol.

Fold 2 (Sub3 held out) is a near-failure: macro F1 collapses to 0.105 because the model predicts Drowsy on the overwhelming majority of frames, which gives it Drowsy recall close to 1.0 but tanks precision on Alert and Low Vigilant. Fold 0 and Fold 1 are usable. This is a direct consequence of D3S only containing three subjects: each fold's training pool is two faces, and if the test subject's facial features fall outside the convex hull of those two, performance falls off a cliff.

4.3.3 Confusion Matrix Analysis

The confusion matrices (see evaluation_reports/confusion_matrices.png) show the same pattern from a different angle. MobileNetV2's Drowsy-to-Alert cells are zero across all three folds, which is what the 0.000 rate in Table 1 reflects. The model's errors are instead concentrated in the Alert-to-Drowsy direction (false positives, where an alert driver is flagged as drowsy). False positives are recoverable: a temporary false alarm is preferable to a missed drowsy event.

ResNet50V2's confusion matrices tell the opposite story: large counts in the Drowsy-to-Alert cells on Fold 0 (121 of 157 Drowsy frames) and Fold 2 (almost no correct Drowsy predictions). This is what the 61.4% Drowsy-to-Alert rate quantifies.

4.3.4 Decision Fusion Results

The decision-fusion module combines CNN softmax with EAR, MAR, and head-pose features. The fusion rules implemented were:

1. EAR/MAR-based override. If the average EAR falls below 0.20, the decision is directly classified as Drowsy regardless of the CNN output. If EAR is below 0.27 or MAR exceeds 0.55, the decision is classified as Low Vigilant.

2. Yawn detection. If MAR exceeds 0.5 for 90 or more consecutive frames (approximately 3 s at 30 FPS), a yawn event is flagged.
3. Head-nod detection. If absolute head pitch exceeds 15°, a head-nod event is flagged.

4. Alert latch. If 3 or more consecutive frames are classified as Drowsy, the visual alert is triggered.

The rules are deliberately conservative: they never down-grade a CNN Drowsy prediction, only escalate ambiguous Low Vigilant frames when independent physiological evidence supports it. Streaming the held-out subject's first 200 test frames through MobileNetV2 plus the fusion rules produces the per-fold outcomes in Table 4.3.4.1.

Fold	Test subject	CNN Drowsy frames	Escalations (EAR-based)	Yawn events	Head-nod flags	Alert triggered
0	Sub1	26	34	0	0	No
1	Sub2	200	0	0	0	Yes
2	Sub3	180	0	0	3	Yes

Table 4.3.4.1: Decision-fusion outcomes on 200 streamed test frames per fold, MobileNetV2.

The EAR-based escalation rule fires 34 times in Fold 0, where the CNN was undercommitted to the Drowsy class but the EAR value dropped below 0.20. Without escalation, Fold 0 would have produced 26 Drowsy frames; with escalation, the fused decision identifies 60 Drowsy frames. This is the fusion module behaving exactly as designed.

The visual alert (3 consecutive Drowsy frames) was triggered on Fold 1 and Fold 2 but not Fold 0, because Fold 0's drowsy frames were interspersed with shorter alert segments that broke the consecutive count. The yawn detector (MAR > 0.5 for 90+ consecutive frames) did not fire on any fold, likely because most yawn frames in D3S show open-mouth states briefer than three seconds of sustained MAR > 0.5.

4.3.5 End-to-End FPS and Latency

The full pipeline (face detection, preprocessing, CNN inference, landmark extraction, fusion, and UI update) was benchmarked on 200 frames per architecture. Table 4.3.5.1 reports per-stage timings.

Stage	MobileNetV2	EfficientNetB0	ResNet50V2
Face detect	659.7 ms	658.2 ms	659.3 ms
Preprocess	0.7 ms	0.5 ms	0.7 ms
CNN inference	49.1 ms	55.8 ms	60.0 ms
Landmarks	2.0 ms	1.9 ms	1.9 ms
Fusion	0.04 ms	0.04 ms	0.04 ms
Total	711.6 ms	716.4 ms	722.0 ms
Mean FPS	1.41	1.40	1.39
Peak RSS	4.4 GB	4.5 GB	4.7 GB

Table 4.3.5.1: Per-stage pipeline latency and throughput, mean over 200 frames (native Windows CPU, TF2.21, no GPU).

End-to-end throughput sits at 1.4 FPS across all three architectures, well below the 15 FPS target stated in the design brief. The cause is unambiguous: dlib's HOG-based face detector accounts for over 90%

of the per-frame budget at 660 ms, while CNN inference is only 49 to 60 ms. The CNN choice barely moves total throughput.

Architecture-level optimisation (TFLite quantisation, lower input resolution, frame skipping) is therefore the wrong place to look for performance gains; the right place is replacing dlib HOG with a faster face detector, or running the pipeline on GPU. On a Colab T4 GPU, CNN inference drops to roughly 5 ms and dlib face detection drops to 30 to 40 ms, putting the full pipeline comfortably above 15 FPS without further changes. The CPU-only number reported here is the worst-case bound, not the deployment target.

4.4 Limitations

The three most material limitations of this system are dataset scale, accuracy ceiling, and runtime throughput on the tested hardware.

1. Dataset scale. D3S contains three subjects. Even with LOSO cross-validation, training on two faces and generalising to a third is the worst-case scenario for face-driven classification: any test subject whose facial geometry, skin tone, or lighting differs noticeably from the two training subjects will produce poor results, which is exactly what we see in MobileNetV2's Fold 2 (Sub3 held out: 25.6% accuracy versus 54.9% and 61.0% on the other folds). The gap between validation accuracy (which peaked at 0.97 to 1.00 because validation frames are drawn from training subjects) and held-out test accuracy (0.42 to 0.52) reaches 50 percentage points. This gap is the genuine generalisation penalty; it is not a training bug. Prior published work on D3S (Nandal et al., 2025; Utari et al., 2025) reports much higher numbers, but neither paper enforces a subject-disjoint split, meaning their reported accuracies reflect the model's ability to recognise the same person across frames rather than generalise to unseen drivers.

2. Accuracy ceiling. Even the best architecture on the best fold (MobileNetV2 on Sub2 at 61.0%) is well below what would be deployable in a real safety system. The cause is not the model: with three subjects, the inter-subject variability dominates whatever within-subject signal the network can extract. No amount of additional augmentation or architectural tuning can manufacture subject diversity that does not exist in the dataset. A meaningful improvement would require expanding D3S or supplementing it with UTA-RLDD (Ghoddoosian et al., 2019), which contains 60 subjects under a predefined 5-fold protocol. We attempted aggressive augmentation, mixup, and test-time augmentation as additional regularisers in a separate ablation; the net effect on safety metrics was negative (Drowsy-to-Alert rate rose from 0.000 to 0.306 for MobileNetV2 under aggressive regularisation), and the version of the system reported here intentionally uses moderate augmentation only.

3. Runtime throughput. 1.4 FPS on CPU is approximately one tenth of the design target. The bottleneck is dlib's HOG face detector, not the CNN. Migration to a faster detector (MediaPipe BlazeFace, OpenCV DNN with a lightweight SSD, or YuNet) or to a GPU runtime would lift this comfortably above the 15 FPS target. Within the current pipeline there is little further headroom on CPU.

A fourth limitation worth noting is the yawn detector's failure to fire on any fold. The MAR threshold of 0.5 with 90 consecutive frames (3 s at 30 FPS) was conservatively set for use on continuous webcam input; D3S's yawn clips are typically too short for sustained MAR exceedance and the model never sees

prolonged yawn states. A shorter MAR window (e.g. 30 frames / 1 s) would activate the detector but would also produce more false positives. This is a calibration trade-off that depends on the deployment environment rather than a fundamental flaw in the fusion logic

5. Discussion and Conclusion

The headline finding of this project is that accuracy alone misranks CNN architectures for safety-critical drowsiness detection. ResNet50V2 had the highest mean accuracy (51.8%) and the lowest fold-to-fold variance (std 0.021) of the three architectures, yet it misclassified 61.4% of actually-drowsy frames as alert. MobileNetV2 had the lowest mean accuracy (47.2%) but a 0.0% Drowsy-to-Alert rate across all three folds. In a deployed system, MobileNetV2's failures are recoverable (false alarms on alert drivers, which can be dismissed) while ResNet50V2's failures are not (drowsy drivers told they are alert). The recommended architecture for this application is therefore MobileNetV2, despite its worse accuracy.

This finding is only visible because of the LOSO evaluation protocol. Random per-frame splits, which are standard in the published D3S literature, allow the same subject's frames to appear in both training and test sets and produce inflated accuracy figures that hide the safety properties of the model. To our knowledge this is the first 3-class LOSO evaluation of D3S. The two existing peer-reviewed papers using D3S (Nandal et al., 2025; Utari et al., 2025) both collapse the dataset to binary drowsy-vs-awake and shuffle frames across subjects, which makes their reported numbers (84% mAP and 90%+ accuracy respectively) inapplicable as comparison points for a subject-independent system.

The decision-fusion module added a measurable safety margin on top of the CNN. On Fold 0, the EAR-based escalation rule increased detected Drowsy frames from 26 to 60 by promoting ambiguous Low Vigilant predictions whose sustained eye-closure was confirmed by the physiological signal. The fusion rules were designed to be conservative: they never downgrade a CNN Drowsy prediction, only escalate, and the visual alert latch (3 consecutive Drowsy frames) debounces the output to prevent single bad frames from triggering false alarms. The yawn detector did not fire on any fold because D3S yawn segments are too short for the 3-second sustained-MAR threshold; this is a calibration issue for continuous video rather than a fundamental design flaw.

End-to-end latency was the project's largest miss against the design brief. The target was 15 FPS; the achieved throughput was 1.4 FPS on native Windows CPU. The CNN choice is not the bottleneck: inference takes only 50 to 60 ms per frame across all three architectures. The bottleneck is dlib's HOG face detector at 660 ms per frame, which consumes 92% of the per-frame budget. Replacing dlib with MediaPipe BlazeFace or YuNet, or moving to a GPU runtime, would push the system above the 15 FPS target without changing the CNN or the fusion module.

5.1 Future Work

Three directions would meaningfully improve this system. First, expanding the training set: D3S's three subjects fundamentally cap LOSO accuracy at the level seen here, and supplementing with UTA-RLDD (Ghoddoosian et al., 2019), which contains 60 subjects under a predefined 5-fold protocol, is the most direct way to improve generalisation to unseen drivers. Second, replacing the face detector: dlib HOG is a 2014-era classical detector and dominates the runtime budget. A modern lightweight detector (MediaPipe BlazeFace, YuNet, RetinaFace-Mobile) would deliver a 15 to 30x speed-up while typically improving accuracy on profile and partially-occluded faces. Third, temporal modelling: this system operates on individual frames and reconstructs temporal context only inside the fusion module. A small

recurrent or transformer head on top of the CNN features could learn drowsiness as a temporal pattern (gradual eye closure, slowing blink rate) rather than as a per-frame state, which is closer to the way drowsiness actually presents in real driving.

5.2 Conclusion

This project delivered a three-class subject-independent drowsiness-detection system on the D3S dataset with a documented decision-fusion module and end-to-end runtime benchmarks. The chosen architecture (MobileNetV2) achieves 47.2% mean LOSO accuracy and a 0.0% Drowsy-to-Alert rate across three folds. The fusion module adds a documented safety margin via EAR-based escalation and a consecutive-frame alert latch. The runtime falls short of the 15 FPS target on CPU due to face-detection cost, not CNN inference. The most generalisable contribution of the work is methodological: LOSO evaluation on D3S reveals trade-offs between accuracy and safety that the published D3S literature has missed by shuffling frames across subjects, and we recommend that any future work on this dataset adopt the subject-disjoint protocol used here.

6. References

Ghoddoosian, R., Galib, M., & Athitsos, V. (2019). A realistic dataset and baseline temporal model for early drowsiness detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 178–187. https://doi.org/10.1109/CVPRW.2019.00027

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Identity mappings in deep residual networks. *Proceedings of the European Conference on Computer Vision (ECCV)*, 630–645. https://doi.org/10.1007/978-3-319-46493-0_38

Nandal, S., Pahal, P., Sharma, B., & Omesh, P. (2025). Real-time driver drowsiness detection using YOLOv8 with whale optimization algorithm. *SAE International Journal of Transportation Safety*, *13*(1), 15–33. https://doi.org/10.4271/09-13-01-0001

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

The Salmon Runs. (2025, September 4). *React vs Streamlit - 2025 comparison* [Video]. YouTube. https://youtu.be/vg1nQd_QJdM?si=ItuRGr-qqZ0v4U17

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *Proceedings of the 36th International Conference on Machine Learning*, *97*, 6105–6114. http://proceedings.mlr.press/v97/tan19a.html

Utari, M., Hendradewa, A., & Bella, M. (2025). Optimized YOLO approach for drowsiness detection in automotive safety: Parameter tuning and facial expression analysis. *International Journal of Transport Development and Integration*, *9*(1), 189–196. https://doi.org/10.18280/ijtdi.090118

Wawrzyński, A. (2023, January 30). *Pros and cons of using Streamlit for simple demo apps*. SoftwareMill. https://softwaremill.com/pros-and-cons-of-using-streamlit-for-simple-demo-apps/

7. Appendices

7.1 Individual Contribution

Team Member 1:

My primary contribution was the design and implementation of the full data pipeline, which formed the foundation for the entire modelling workflow. This included loading and auditing the D3S dataset across all three subjects, remapping the four original expression states to the project's three-class scheme, and implementing a leave-one-subject-out cross-validation protocol to eliminate the data leakage present in earlier versions of the project. I built the face detection and ROI cropping pipeline using dlib's HOG detector with a Haar Cascade fallback, computed balanced class weights per fold to handle class imbalance, and developed the EAR, MAR, and head pose extraction module using dlib's 68-point shape predictor for use in the 2nd team member's decision fusion component. I also constructed the reusable tf.data preprocessing pipelines with a parameterised preprocessing function, allowing all three CNN backbones to share the same pipeline infrastructure without duplication. The pipeline outputs, including fold split definitions and class weights, were exported as JSON files for the 2nd team member's training sections to consume directly.

For the written report, I authored the Abstract and the Introduction and Background section in full, covering the problem statement, motivation, application scope, and a detailed description of the dataset including its class distribution, label remapping rationale, and the subject-independent evaluation protocol. I also contributed to Section 3, which covers the system architecture and design, including the creation of the system flowchart illustrating the end-to-end pipeline from user authentication through to real-time drowsiness detection and session reporting. I also handled the overall formatting of the report, ensuring consistent structure, heading hierarchy, and professional presentation throughout.

Team Member 2:

My contributions covered the training pipeline, evaluation framework, decision-fusion module, runtime benchmarking, and Sections 4 and 5 of this report.
For training, I implemented the two-phase transfer-learning pipeline for all three CNN backbones (MobileNetV2, EfficientNetB0, ResNet50V2) using the Keras functional API. Each backbone was wrapped with training=False at the call site so BatchNormalization layers remained in inference mode during Phase 2 fine-tuning, preserving the ImageNet-pretrained moving statistics. I configured the per-architecture hyper-parameters (unfreeze depth, Phase 2 learning rates from 1e-4 to 1e-5), label smoothing of 0.1, per-fold class weights, and the ReduceLROnPlateau and EarlyStopping callbacks. All nine model-fold combinations were trained under the subject-disjoint LOSO protocol.
For evaluation, I built the per-class precision/recall/F1 reporter, macro F1 aggregator, confusion-matrix generator with Drowsy-to-Alert highlighting, and one-vs-rest ROC AUC computation. The architecture comparison routine produced the project's headline finding: ResNet50V2's higher mean accuracy masks a 61.4% Drowsy-to-Alert false-negative rate, whereas MobileNetV2's lower accuracy comes with a 0.0% rate across all folds.

For decision fusion, I implemented the streaming DecisionFusion class combining CNN softmax with EAR, MAR, and head-pose features. The class encodes all four fusion rules: EAR-based escalation of

ambiguous Low Vigilant predictions, sustained-MAR yawn detection, head-nod detection on absolute pitch, and the configurable consecutive-frame alert latch. For runtime analysis, I built the end-to-end FPS benchmark with per-stage latency profiling and peak memory tracking, which produced the diagnosis that dlib's HOG face detector, not the CNN, dominates the per-frame budget.

Finally, I authored Sections 4 and 5 of this report covering experimental results, the safety-versus-accuracy finding, limitations, discussion and recommendations for future work.

Brunda Varshini Katapalli:

My primary responsibility within the team was the design and implementation of the whole of Section 3 of the project report, which encompassed GUI and Backend integration of the VigiDrive system. I was solely responsible for developing the complete frontend application using React, building all screens from the landing page through to the session end summary, including the authentication flow, camera permission handling, face alignment verification and the live monitoring interface with real-time landmark visualisation and Grad-CAM heatmap overlay.

On the backend, I developed the FastAPI inference server in Python, integrating the pre-trained MobileNetV2 model with dlib facial landmark detection to compute real-time Eye Aspect Ratio, Mouth Aspect Ratio, and head pose estimation on every incoming webcam frame. I implemented the Decision Fusion engine, which combines CNN predictions with direct EAR and MAR threshold rules to produce robust drowsiness classifications across varying environments. I also managed the full deployment pipeline, configuring the backend to run on Google Colab with a persistent ngrok tunnel and resolving Keras version compatibility issues to ensure the model loaded correctly in the Colab environment.

Beyond technical contributions, I took on the project management role within the team, coordinating task allocation, tracking progress, and ensuring all members remained on schedule throughout the project lifecycle. I consistently followed up with teammates to confirm deliverables were completed on time and facilitated communication to keep the team aligned ahead of submission deadlines.

In addition to the implementation work, I designed the end-to-end system flowchart, mapping the complete process flow from user authentication through to session completion, including all decision branches, backend inference steps, and alert trigger logic.

7.2 Individual Contribution Split

Team Member Name	Percentage
Team Member 1	33%
Team Member 2	33%
Brunda Varshini Katapalli	33%

All the team members have discussed and agreed on the above individual contribution to the project.